

Smart City Traffic Intelligence: Final Capstone Report

Metro Interstate Traffic Volume · I-94 westbound (MnDOT ATR 301), Oct 2012 – Sep 2018 Parts 1–3: data analytics → reproducible Python pipeline → machine learning, MLOps and responsible AI

Accident data statement: no real accident dataset was sourced. The classification task uses the **documented proxy label** defined in the brief (High/Severe congestion during severe or low-visibility weather). It demonstrates the workflow only and must not be read as accident prediction.

1. Executive summary

- **Traffic on this corridor is overwhelmingly a function of time.** Hour of day and day type explain most of the variance. The time-and-weather regressor reaches **MAE 217 vehicles/hour (R^2 0.966)** on unseen 2018 data. Adding the previous hours' volumes cuts next-hour error to **MAE 139 (R^2 0.988)**.
- **Weather matters far less than intuition suggests, except snow.** Snow is also where every model is least accurate (MAE 2.7× the average, with volumes over-predicted).
- **A small, well-engineered gradient-boosting model beat the LSTM** (MAE 139 vs 156) at a fraction of the compute.
- **The proxy "risk" classifier scores almost perfectly (ROC AUC $\approx$ 1.0), but for the wrong reason.** A two-condition rule gets F1 0.885, and SHAP attributes 61% of the model's decisions to the weather columns that define the label.
- The solution is packaged as an **MLflow-tracked, versioned, FastAPI-served** system with a **travel-time recommender** and **PASS/ALERT monitoring**. That monitoring correctly flagged the anomalous winter months of 2018 and two injected incidents.

2. Parts 1 and 2 recap (foundation)

Part 1 – SQL and Power BI ([part1_data_analytics/](#)). SQL queries on the raw 48,204 rows found clear commuter peaks (7–9 AM, 4–6 PM), a weak temperature–volume correlation ($r \approx 0.13$), and weather that is roughly independent of congestion (odds ratio for clear vs cloudy = 0.74). A Power BI dashboard presented daily and hourly trends, weather impact and KPIs.

Part 2 – Python pipeline ([part2_python/](#)). A logged, tested pipeline validated the schema and cleaned the data: * standardised labels (1,730 rows); * removed 17 duplicates and collapsed 7,612 repeated hourly weather rows; * imputed 0 K temperatures and a 9,831 mm rainfall reading with the monthly median; * imputed 47 sensor-outage counts by (hour, day type) median.

The result was 40,575 clean hourly rows. It then engineered 38 features and a quartile congestion target, and produced seven figures and a CLI. Its key insight carried into Part 3: once time of day is controlled for, only snow clearly suppresses traffic (–8% on workday daytimes).

3. Part 3 methodology

3.1 Data preparation and evaluation protocol

- **Reuse, not duplication:** Part 3 imports Part 2's cleaning output and weather definitions. `features.py` is the single feature builder for **both training and serving**, which removes train/serve skew.
- **Common feature set (32 features):** hour, day of week, month, weekend, **holiday** (propagated to the whole date), rush hour, **sin/cos encodings of hour, day of week and month**, temperature (°C), log rainfall, snow, cloud cover, precipitation, low-visibility, severe and freezing flags, an ordinal weather severity, and **11 one-hot weather categories**.
- **Labels:** `congestion_category` from traffic-volume quartiles with inclusive boundaries ($\text{Low} \leq 1,255 < \text{Medium} \leq 3,430 < \text{High} \leq 4,952 < \text{Severe}$). `high_risk` proxy = High/Severe congestion AND (Thunderstorm/Squall OR Mist/Fog/Haze/Smoke): 1,927 positive hours (4.75%).
- **Time-based split, no shuffling:** train Oct 2012–Dec 2016 (25,329 h), validation 2017 (8,713 h) for model selection, test Jan–Sep 2018 (6,533 h), used once for reporting. Each final model is refitted on train + validation.

3.2 Reproducibility and logging

`python run_all.py --fresh-mlflow` rebuilds every result in ≈ 3.5 minutes on a laptop CPU. Every module uses `logging.getLogger(__name__)`, and handlers are configured only by the entry points. The log levels mean: * **INFO:** milestones. * **WARNING:** data changes, drift and alerts. * **ERROR:** failures, with the traceback included.

Logs are in `logs/`. Randomness is seeded, and 15 pytest tests cover features, labels, monitoring rules, the recommender and the API.

4. Task 1 – Supervised learning

Figures: `figures/task1_supervised/`

Classification: proxy high-risk (test set, threshold 0.5)

Model	Accuracy	Precision	Recall	F1	ROC AUC	PR AUC
Rule reference, no ML (risky weather AND 06–18h)	0.986	0.810	0.975	0.885	0.981	0.791
Logistic regression (baseline, balanced)	0.989	0.833	1.000	0.909	0.999	0.981
Random forest	0.995	0.919	0.992	0.954	1.000	0.993
HistGradientBoosting v1	0.995	0.928	0.983	0.955	1.000	0.993
HistGradientBoosting v2 (champion)	0.995	0.917	0.994	0.954	1.000	0.994

The champion was selected on validation PR AUC (0.997), which suits a 4.75%-prevalence label. Its F1-optimal threshold on validation is 0.834, giving test precision 0.935, recall 0.977 and F1 0.956 (confusion matrix: 346 TP, 24 FP, 8 FN).

Interpretation: the tree ensembles mainly add *precision* over the linear baseline, learning the non-linear "congestion needs daytime" interaction. Every metric is near-perfect because the label is computed from the inputs. The rule reference shows that most of the signal needs no learning at all (see §6 and the bias report).

Regression: hourly traffic volume (test set)

Model	Val MAE	Test MAE	Test RMSE	Test R ²
Ridge regression (baseline)	570	550	709	0.871
Random forest	231.8	218	369	0.965
HistGradientBoosting v1 (champion)	231.5	217	364	0.966

Model	Val MAE	Test MAE	Test RMSE	Test R ²
HistGradientBoosting v2 (tuned)	240	221	367	0.966

Interpretation: the linear model cannot express the two-peak daily profile, even with cyclical features. Its errors exceed 1,000 vehicles/hour at 06–08. Tree ensembles cut error by **60%**. Random forest and HGB v1 are tied on validation; HGB v1 was chosen and is also 3× faster to train and far smaller (0.3 MB compressed). "Tuning" (v2) did not help, a reminder that more capacity is not automatically better.

5. Task 2 – Unsupervised learning

Figures: `figures/task2_unsupervised/`

K-means clustering of traffic conditions

Clustering used standardised hour (sin/cos), weather severity and traffic volume. Silhouette was flat between $k = 2$ and 8 (0.35–0.39). The rule "smallest $k \geq 3$ within 0.01 of the best silhouette" selected **$k = 4$** (silhouette 0.386).

Cluster	Share	Mean volume	Profile	Interpretation
0 Low traffic, night	32%	1,037	typical hour $\approx$ 02:00, clear, 0% adverse weather	Overnight baseline from about 22:00 to 05:00; almost no severe congestion.
1 Moderate traffic, adverse weather, all hours	8%	3,006	100% severity ≥ 2 (snow or freezing precipitation), mean 0.8 °C	A weather regime rather than a time regime: bad-weather hours cluster together wherever they fall in the day.
2 Heavy traffic, afternoon–evening	28%	4,196	typical hour $\approx$ 18:00, 30% severe congestion	PM peak and its evening decline.
3 Heavy traffic, morning–midday	32%	4,839	typical hour $\approx$ 10:00, 48% severe congestion , 11% proxy risk	AM peak through midday: the most congested regime.

Association rules (Apriori, min support 0.5%, min confidence 50%, non-redundant, ranked by lift)

Hours were discretised into time of day, day type, weather group and temperature band. The strongest rules for each level: * "During the **night (00–05) at weekends** with freezing temperatures, congestion is **Low 95%** of the time, **3.8×** the base rate (914 hours)." * "During the **morning peak (06–09) on weekdays** with cold temperatures, congestion is **Severe 92%** of the time (3.7×, 942 hours)." * "During the **afternoon peak (15–18) at weekends** with warm temperatures, congestion is **High 92%** of the time (3.7×)." * "During the **evening (19–23) on public holidays**, congestion is **Medium 82%** of the time (3.3×)."

Meaning: every high-lift rule is anchored on *time of day and day type*. Weather and temperature appear only as refinements (e.g. rain moves weekend evenings into Medium). This independently confirms Parts 1–2 and the SHAP results.

6. Task 3 – Deep learning with explainability

Figures: `figures/task3_deep_learning/`, `figures/task3_explainability/`

Model. A PyTorch LSTM reads the previous **24 hours** (volume, cyclical calendar, holiday, temperature, weather severity, precipitation, clouds). Its encoding is concatenated with the *known* next-hour calendar and forecast weather to predict next-hour volume. Training used Huber loss, Adam, gradient clipping, a learning-rate schedule and early stopping on 2017 data.

Sequence data. 2,452 of 2,588 gaps last ≤ 3 hours, and naively dropping broken windows discarded 29% of the data. Short gaps were linearly interpolated as *inputs only*; targets are always real observations. This gave 39,171 windows (24,045 train / 8,617 validation / 6,509 test).

Next-hour model (identical test windows)	MAE	RMSE	R ²	Train time
Persistence (last hour)	588	813	0.830	–
Seasonal naive (same hour yesterday)	566	1,031	0.727	–
Task 1 regressor (no recent history)	217	363	0.966	0.6 s
LSTM v1 (1 layer, 32 units, 8.5k params), champion	156	233	0.986	18 s
LSTM v2 (2 layers, 64 units, 57.7k params)	158	242	0.985	80 s
HistGB + lag features (explainable surrogate)	139	217	0.988	1.6 s

Explainability method and why a surrogate. SHAP on a recurrent network produces attributions per time step and input, and requires thousands of forward passes. As the brief permits, the LSTM is explained through a **comparable HistGradientBoosting model trained on the same next-hour problem, the same samples and equivalent information** (lags 1, 2, 3 and 24 plus 24-hour rolling mean and std, calendar and weather). The two reach similar accuracy on identical windows, and TreeSHAP gives exact Shapley values for the tree model.

SHAP findings: * **Next-hour surrogate:** recent history carries 58% of attribution (`lag_1` 49%, `lag_24` 3%), then time of day (`hour_cos` 22%, `hour` 8%, `is_rush_hour` 6%). Weather features are individually under 1–2%. The hour dependence plot shows positive contributions from 06:00 to 18:00, strongest on weekday peaks, and negative contributions at night. * **Task 1 regressor (no lags):** `hour_cos` 47%, `hour` 19%, `is_rush_hour` 14%, `day_of_week` 6%. * **Local explanation (Thu 11 Jan 2018 08:00, snow):** the model predicted 5,124 against 4,475 actual. It was pushed up by last hour (+968), time of day (+466) and rush hour (+355), and down only slightly by weather severity (–49). This shows *why* snow errors are large: the model has learnt little snow penalty. * **Proxy classifier:** `is_low_visibility` 45%, `hour_cos` 22%, `weather_Thunderstorm` 9%, `weather_severity` 7%. **61%** comes from label-defining weather features, which is quantitative evidence that the classifier re-learns the label rule.

Conclusion: the LSTM clearly learns sequential behaviour, cutting error by more than 70% against the naive baselines and by 28% against the history-free model. However, the gradient-boosting surrogate is more accurate, 11× faster to train and explainable. For this data size and horizon it is the model I would deploy.

7. Task 4 – Advanced technique: MLflow experiment tracking and model registry

Why chosen	The project trains many candidates across five experiments. Without tracking, comparisons would rely on console output and memory. MLflow connects directly to the MLOps tasks: the same registry versions and <code>champion</code> alias are what the API, monitoring and version history consume.
-------------------	--

How implemented	<code>mlflow_utils.py</code> uses a SQLite backend (<code>mlflow/mlflow.db</code>), which enables the Model Registry, with local artifacts in <code>mlflow/mlartifacts/</code> . There are five experiments: classification, regression, unsupervised, LSTM and monitoring. Each run logs hyperparameters, validation and test metrics, training time, tags (task, algorithm, label type) and the git commit. Scikit-learn models are logged with an inferred signature and input example; LSTMs with <code>mlflow.pytorch</code> , including per-epoch loss and validation-MAE curves. Every candidate is registered as a new model version . The best validation candidate receives the champion alias plus a selection-reason tag and is exported to <code>models/</code> . Exports: <code>reports/metrics/mlflow_runs_summary.csv</code> (26 runs) and <code>reports/MODEL_VERSIONS.md</code> .
Value added	Side-by-side comparison of 10 registered versions across three models. Selection is auditable (which version, why, trained on what, from which commit). Rolling back is a single alias change. Monitoring runs are logged next to training runs, so a performance history exists. It also exposed issues: the tied RF/HGB validation MAE, and the LSTM v2 costing 4.4× the energy for no gain.
Limitations	The local SQLite store has no authentication or concurrency, and records absolute artifact paths , so artifacts must be regenerated on another machine (the database and CSV exports remain readable). MLflow warned that pickled models can execute code when loaded ; this is acceptable for trusted local artifacts, but production would need signed artifacts or a safe format. MLflow 3.15 declares <code>pandas<3</code> while the project ran on pandas 3.0.5; everything worked, but it is a supply-chain risk to pin. Tracking records <i>what</i> happened, not <i>whether it was right</i> , so governance still needs human review.

Browse the tracked runs with `mlflow ui --backend-store-uri sqlite:///mlflow/mlflow.db --port 5000` from `part3_machine_learning/`.

8. Task 5 – Travel-time recommendation system

Code: `recommendation_system/recommender.py` · **Examples:** `recommendation_system/recommendation_examples.md`

Because the data covers one corridor, the system recommends **when** to travel, not which route. For a date, an acceptable time range, a trip-window length and forecast weather, it: 1. builds a feature row per candidate hour (day type, holiday calendar, cyclical time, weather; temperature defaults to the historical median for that month and hour) and predicts volume with the champion regressor; 2. converts predictions to congestion categories and scores the proxy-risk classifier as a *caution flag*; 3. ranks every contiguous window by mean predicted volume, deprioritising flagged windows, and compares the best one with the busiest window and with the historical median for that day type and hour; 4. writes a plain-language recommendation with alternatives and weather-specific advice.

Example output (weekday, snow, 2-hour window, 07:00–20:00):

"For a weekday journey on Tuesday 16 January 2018 in snow, consider travelling between 18:00 and 20:00, when traffic is forecast at about 3,446 vehicles per hour (High congestion). That is roughly 35% lighter than the busiest option in your range, 16:00–18:00 (~5,304). ... Good alternatives: 10:00–12:00 (~3,755) ... Snow lowers daytime volumes by about 8% historically but makes each journey slower and riskier; allow extra time."

Other scenarios: a clear weekday with a 06:00–21:00 range recommends 20:00–21:00 (56% lighter than 07:00–08:00), with 10:00–11:00 as the best daytime alternative; a mist morning (06:00–12:00) recommends 10:00–11:00 and adds a proxy-risk caution for the peak hours; a public holiday (4 July) recommends 08:00–09:00 at around 2,000 vehicles/hour, 46% lighter than the afternoon. The same engine powers the CLI (`python -m recommendation_system.recommender --date ...`) and the API's `/recommend` endpoint.

9. Task 6 – MLOps and deployment simulation

6.1 Model versioning. `reports/MODEL_VERSIONS.md` lists 10 versions across three registered models, with hyperparameters, validation and test metrics, training time and status. Champions: classifier **v4** (HGB v2), regressor **v3** (HGB v1), LSTM **v1**. Each served model carries a metadata card (`models/*.json`).

6.2 Experiment tracking. See §7. There are 26 MLflow runs across 5 experiments, all with params, metrics, artifacts and versions.

6.3 Deployment (FastAPI). `deployment/app.py` serves: * GET `/health` and GET `/model-info` (versions, metrics, proxy disclaimer); * POST `/predict/traffic-volume`, POST `/predict/risk` and POST `/recommend`.

Inputs are validated with pydantic (ranges, weather enum). Invalid requests return HTTP 422 and are logged as a WARNING, and every prediction is appended to an audit log. It was verified both in-process and over HTTP with uvicorn (`python -m deployment.serve`): all 8 demo calls returned the expected status codes (`reports/api_demo_responses.json`, `logs/api.log`). For example, 08:00 on a clear weekday gives $\approx 6,000$ vehicles/hour (Severe), and the same hour in mist gives proxy risk $p = 0.996$ against 0.003 in clear weather.

6.4 Monitoring. `monitoring/monitoring.py` replays Jan–Sep 2018 as monthly production batches against the champion regressor: * **Feature drift:** PSI per feature against the **same calendar month of 2017**, with cloud cover and weather treated as categories. * **Prediction error drift:** batch MAE against validation MAE, plus bias. * **Data integrity:** the share of values outside the training range.

Two lessons shaped the design: 1. The first version compared against *all* 2012–2017 data and **alerted on 9 of 9 months**, because the weather feed began rounding cloud cover to five values in 2017. A recent, seasonal reference fixed this false-alarm storm. 2. 2018 was a genuinely anomalous weather year (record-cold April, record-warm May), so temperature drift alone is not a reason to page anyone.

The final **tiered policy** raises an **ALERT** only on model impact ($\text{MAE} > 1.3\times$ validation, or $|\text{bias}| > 10\%$) or broken data ($> 1\%$ out of range). Feature drift is recorded as a **watch** note.

6.5 Alerting and dashboard. `monitoring/dashboard.html` shows a status banner and per-batch PASS/ALERT badges with findings (charts: `figures/task6_monitoring/`).

Batch	Status	Reason
Jan 2018	ALERT	Error drift: $\text{MAE } 307 = 1.33\times$ validation (cold January)
Apr 2018	ALERT	Error drift: $\text{MAE } 303 = 1.31\times$ (record-cold April, blizzard); temperature PSI 1.22
Feb, Mar, May–Sep 2018	PASS	Errors $0.68\text{--}1.05\times$; drift noted as watch where PSI > 0.25
SIM: sensor under-count ($\times 0.65$)	ALERT	MAE $5.09\times$, bias $+55\%$
SIM: temperature feed in °F	ALERT	100% of temperatures outside training range

Current status: PASS / Normal (latest batch Sep 2018). The dashboard also shows the two alerts raised earlier in the period.

10. Task 7 – Responsible and sustainable AI (summary)

The full analysis is in `reports/BIAS_FAIRNESS_REPORT.md`. * **Coverage:** a single sensor and direction; 2014 and 2015 have only 51% and 41% of hours; ≈ 10 -month gap; 53 holiday days; rare Fog, Smoke and Squall. * **Proxy label:** circular (a rule gets F1 0.885; 61% of SHAP attribution comes from label-defining weather); structurally zero risk in clear, rain, snow and night hours; 63% of positives are Mist; keeping secondary weather conditions would add **69%** more positives. * **Uneven errors:** snow MAE $2.71\times$ (over-predicting by +334), holidays $1.79\times$, PM peak $1.42\times$, winter $1.28\times$. Classifier false-positive rate 23% in thunderstorms; evening recall 0.33; fairness is undefined wherever the proxy has no positives. * **Governance:** validation gates, registry-based approval and rollback, audit

logs, disclaimers, monitoring with human ownership, and a shadow-mode period before any operational use. The proxy risk output must never drive decisions on its own. * **Sustainability:** all 10 registered versions trained in 105 s (≈ 0.9 Wh, ≈ 0.4 g CO₂). LSTM v2 used 76% of that energy for no accuracy gain. The deployed champion is 0.3 MB and predicts in ≈ 4 μ s on a CPU.

11. Limitations and future work

1. Obtain real crash data (e.g. MnDOT) and multi-sensor or network data to enable genuine risk modelling and route choice.
 2. Add holiday-proximity, snow-accumulation and road-condition features, and evaluate snow and holiday segments as release gates.
 3. Keep multi-condition weather; quantify forecast-weather uncertainty in recommendations (currently the user supplies the weather).
 4. Productionise MLflow (remote tracking server, signed artifacts, approval workflow) and schedule monitoring with automated retraining triggers.
 5. Measure real-world impact of recommendations (peak spreading vs induced demand) before scaling.
-

Appendix – Where to find things

Deliverable	Location
Models, notebooks and scripts	<code>part3_machine_learning/*.py</code> , <code>models/</code> , <code>notebooks/part3_results_walkthrough.ipynb</code>
MLflow logs	<code>mlflow/mlflow.db</code> , <code>mlflow/mlartifacts/</code> , <code>reports/metrics/mlflow_runs_summary.csv</code> , <code>reports/MODEL_VERSIONS.md</code>
Deployment simulation	<code>deployment/</code> (FastAPI app, server launcher, demo client), <code>reports/api_demo_responses.json</code>
Monitoring and alerting	<code>monitoring/monitoring.py</code> , <code>monitoring/dashboard.html</code> , <code>monitoring/monitoring_report.json</code>
Recommendation engine	<code>recommendation_system/</code> (engine, CLI, examples), <code>models/serving_reference.json</code>
Responsible AI report (bias, fairness, governance, sustainability)	<code>responsible_ai_report.pdf</code> (source: <code>reports/BIAS_FAIRNESS_REPORT.md</code>)
Logs	<code>logs/run_all.log</code> (full pipeline), <code>logs/api.log</code> , <code>logs/api_demo.log</code>